Ensemble Methods

TP – Risk certificates for ensemble methods

M2 Computer Science – MALIA

Guillaume Metzler
Institut de Communication (ICOM)
Université de Lyon, Université Lumière Lyon 2
Laboratoire ERIC UR 3083, Lyon, France

guillaume.metzler@univ-lyon2.fr

Abstract. In this practical session you will compute PAC-Bayesian risk certificates for random forests, Gibbs posteriors, boosted ensembles and linear classifiers on the datasets of the course, learn the weights of majority votes by minimizing these certificates, use them for model selection, and compare them with the classical hold-out certificate. The session relies on notebooks 1 to 5 and on the module `pbtools.py`. The work is organized in **four sessions of three hours**, each starting on a new page with its goals, the sections of the lecture notes to reread, a suggested timing and the results to hand in; parts are numbered by session (Part 2.3 is the third part of Session 2). The sessions build on each other, but the parts inside a session are independent after its first part and can be split among the members of a group. Parts marked (⋆) are longer.

Session	Theme	Parts
1	Certifying a classifier and a forest	warm-up on the kl, selection bias, OOB certificates, trees and bootstrap size
2	Self-bounding learning with forests	learning the weights, exact kl minimization, strength vs diversity, model selection
3	Gibbs posteriors and hard cases	temperature of a Gibbs posterior, hold-out vs self-bounding, imbalance, boosting
4	Linear classifiers and synthesis	PBGD and Monte Carlo certificates, data-dependent priors, synthesis, report

Session 1 – Certifying one classifier and a random forest

Goal. By the end of the session you should be able to compute a test-set bound by hand and with the kl inversion, understand selection bias, and compute the first-order, tandem and C-bound certificates of a random forest from its out-of-bag samples.
To reread before the session. Sections 2, 5 and 7.2 of the notes; notebooks 1 and 3.
Timing (3 hours).

Part 1.1	20 min
Part 1.2	50 min
Part 1.3	60 min
Part 1.4	50 min

To hand in at the end of the session. Your notebook, with the table of Part 1.3 on your 10–15 datasets and the table of Part 1.4.

Part 1.1 – Setting up

- Download the datasets (`datasets.zip`) and the loading script `prepdata.py` of the first practical session, and the module `pbtools.py`. The function `data_recovery(name)` returns a pair `(X, y)` with labels in $\{0, 1\}$: convert them to $\{-1, +1\}$ and standardize the features.
- Read `pbtools.py`: every function is short. It provides the binary kl and its inversions (`kl_bin`, `kl_inv_upper`, `kl_inv_lower`), the classical bounds (`bound_seeger`, `bound_mcallester`, `bound_catoni`, `bound_lambda`), a random forest keeping track of its out-of-bag samples (`BaggedTrees` with `oob_losses`, `oob_tandem`, `oob_disagreement`), the majority-vote certificates (`fo_bound`, `tnd_bound`, `cbound_pb`), two optimizers (`optimize_fo`, `optimize_tnd`) and the gradient of the kl-form certificates (`certificate_and_grad`).
- Use **at least 10 datasets**, balanced and imbalanced, for instance `australian`, `balance`, `bupa`, `german`, `heart`, `iono`, `pima`, `sonar`, `wdbc`, `spambase` (balanced group) and `abalone8`, `pageblocks`, `satimage`, `segmentation`, `yeast3` (imbalanced group).

> **Good practice.** Fix all the random seeds, and write one function per experiment that returns a table: you will rerun them many times. Before computing a certificate, write down on paper *which examples* each voter has seen: this single question avoids most of the mistakes of this session.

Part 1.2 – Warm-up: inverting the kl and certifying one classifier (Section 2)

1. Implement the upper inversion $\overline{\mathrm{kl}}^{-1}(\hat{q}, \varepsilon)$ by bisection (Algorithm 1 of the notes) and check it against `kl_inv_upper`. How many iterations are needed for a precision of 10^{-9}?
2. For $m \in \{100, 1000\}$, $\delta = 0.05$, $\varepsilon = \ln(1/\delta)/m$ and $\hat{q} \in \{0, 0.01, 0.05, 0.1, 0.3, 0.5\}$, compare three upper confidence bounds on the risk of a *fixed* classifier: Hoeffding $\hat{q} + \sqrt{\varepsilon/2}$, the kl inversion, and the refined Pinsker relaxation $\hat{q} + \sqrt{2\hat{q}\varepsilon} + 2\varepsilon$. When does the kl make a difference?
3. On `australian`, split the data in two halves. Learn a decision tree of depth 3 on the first half and compute the Hoeffding and kl test-set bounds of its risk on the second half.
4. Now learn trees of depths $1, \ldots, 10$ on the first half, keep the one with the smallest error on the second half, and compute its kl bound as if it had been fixed in advance. Why is this "certificate" not valid? Correct it.

Part 1.3 – Certificates of a standard random forest (Sections 5 and 7.2)

For each dataset, repeat 5 times: split the data into 70% training / 30% test (stratified) and learn a `BaggedTrees` forest of 100 trees on the training set with bootstrap samples of size $m/2$.

1. Compute the test risk of the uniform majority vote, the test Gibbs risk and disagreement, and the three OOB certificates (first-order, tandem, C-bound) of the uniform posterior, with $\delta = 0.05$. Report also the sizes n_1 (smallest OOB set) and n_2 (smallest intersection of two OOB sets).
2. Check that the certificates are always above the test risk. Which bound is the tightest, on which datasets? Relate your answer to the disagreement of the trees (Proposition 5.7) and to n_1, n_2.
3. Compare the test risk of `BaggedTrees` with `sklearn.ensemble.RandomForestClassifier` (full bootstrap). How much accuracy do we lose by using bootstrap samples of size $m/2$?

Part 1.4 – Number of trees and size of the bootstrap samples (Section 7.2)

On `spambase` (3 splits), learn forests with $n \in \{10, 50, 100, 200\}$ trees and bootstrap samples of size $k = f\,m$ with $f \in \{0.3, 0.5, 0.8, 1\}$.

1. Before running anything, predict how n_1 and n_2 depend on f (compute $(1 - 1/m)^{fm}$ and its square), and whether they depend on n.
2. Report the test risk of the uniform vote, n_1, n_2 and the FO and TND certificates. Which configuration gives the best vote? the best certificate? Is there a configuration that is good for both?